

1 Отчёт по задаче 3.1: объединение типов запросов DiGr DSL

1.1 Три вида запроса в исходной системе

Источники: docs/dsl_grammar.ebnf, src/dsl/model/query_ast.py, src/dsl/execution/query_results.py, src/dsl/actors/execution/execution_coordinator_actor.py исходного архива (плюс task-1/dsl_grammar.rbnf и task-2/dsl_report.tex).

База у всех запросов была общей: `query = context_query | find_query | distance_query`. Но дальше каждый вид рабтал не очень: своя продукция, свой порядок разделов, свой класс AST, свой класс результата.

	FIND	CONTEXT	DISTANCE
Заголовок	FIND entity	CONTEXT entity[N] FOR patterns	DISTANCE sel TO sel
Порядок хвоста	WHERE?, WITHIN*, RETURN?	WITHIN*, WHERE?, RETURN?	WITHIN*, LIMIT_PAIRS?, RETURN?
Класс AST	FindQuery	ContextQuery	DistanceQuery
Класс результата	FindQueryExecutionResult	ContextQueryExecutionResult	DistanceQueryExecutionResult
Ключ type в JSON	find_query_execution_result	context_query_execution_result	distance_query_execution_result

У FIND порядок WHERE/WITHIN был обратным по сравнению с CONTEXT и DISTANCE. Общего понятия "хвоста запроса" не было вообще: парсер трижды писал разбор WITHIN/WHERE/RETURN, один раз на каждый вид запроса, почти одинаковым кодом.

1.2 Единая грамматика

Рассматривал два варианта: одна продукция с разными заголовками по ключевому слову, вообще убрать ключевые слова и определять вид запроса по структуре. Взял первый вариант — он почти не меняет синтаксис запросов и убирает тройное дублирование хвоста.

Полная грамматика — docs/dsl_grammar.rbnf. Верхний уровень:

```
< апрос> ::= < аголовок запроса> < вост запроса> [ ";" ] ;

< аголовок запроса> ::= < аголовок CONTEXT>
                        | < аголовок FIND>
                        | < аголовок DISTANCE> ;

< вост запроса> ::= { < граничение WITHIN> }
                  [ < словие WHERE> ]
                  [ < лок LIMIT_PAIRS> ]
                  [ < лок RETURN> ] ;
```

Заголовок остался специфичным для ключевого слова, потому что там реально разное количество операндов: один селектор у FIND, спецификация окна и список паттернов у CONTEXT, пара селекторов у DISTANCE. Хвост — одна продукция на все три вида, без исключений.

Порядок хвоста пришлось нормализовать. Раньше `FIND` требовал `WHERE` перед `WITHIN`, а `CONTEXT` и `DISTANCE` — наоборот. Раз хвост теперь один на всех, пришлось выбрать один порядок: `WITHIN*`, `WHERE?`, `LIMIT_PAIRS?`, `RETURN?`. Это единственное место, где старые запросы вида `FIND entity WHERE ... WITHIN ...` перестают парситься — нужно писать `FIND entity WITHIN ... WHERE ....` Тест на это добавлен в `tests/dsl/test_lexer_and_parser.py` (`test_dsl_parser_builds_find_query_with_within_before_where`).

LIMIT_PAIRS теперь синтаксически доступен не только DISTANCE. Грамматика не проверяет, что `LIMIT_PAIRS` пишут только у `DISTANCE` — раз хвост общий, запрет пришлось бы делать либо отдельной продукцией (опять дублирование), либо проверкой уже после разбора. Выбрал второе: то же самое и так уже было с именами сущностей — грамматика их не ограничивает, проверяет валидатор.

1.3 Единый узел AST: Query

`src/dsl/model/query_ast.py:131--152`. Вместо трёх датаклассов один:

```
class Query(Serializable):
    kind: str
    source: Selector
    target: Selector | None = None          # только DISTANCE
    window: CountConstraint | None = None  # только CONTEXT
    patterns: list[Pattern] | None = None  # только CONTEXT
    within: list[WithinConstraint] = field(default_factory=list)
    where: Expression | None = None
    limit: PairLimit | None = None         # только DISTANCE
    returns: list[ReturnItem] | None = None
```

Поле `source` есть у всех трёх, но означает разное: у `FIND` — выбираемая сущность, у `CONTEXT` — базовая сущность окна, у `DISTANCE` — левый селектор пары. У `CONTEXT` `source.predicate` всегда `None`: скобки после сущности в заголовке `CONTEXT` — это не предикат, а ограничение длины окна, оно лежит отдельно, в `window`. Не сразу это заметил при переносе кода, пришлось перечитывать грамматику ещё раз.

Поля `target`, `window`, `patterns`, `limit` нужны только одному конкретному `kind`, у остальных двух так и остаются `None`.

Парсер повторяет структуру грамматики: `parse_query_head()` (строка 90) смотрит на ключевое слово и возвращает частично заполненный `Query`, `parse_query_tail()` (строка 125) дозаполняет его общими полями.

1.4 Единый формат результата

`src/dsl/execution/query_results.py`. Было три класса результата (`FindQueryExecutionResult`, `ContextQueryExecutionResult`, `DistanceQueryExecutionResult`) с разными ключами в `to_dict()`. Стал один — `DslQueryExecutionResult`. Конверт результата один и тот же для всех трёх видов запроса:

```
{
    "type": "dsl_query_execution_result",
    "kind": "find" | "context" | "distance",
    "count": <int>,
    "returns": [...],
    "items": [...],
    "stats": {...}    // присутствует, если "stats" запрошен в RETURN
```

```
}
```

Вид запроса теперь виден по полю `kind`, а не по имени класса результата. Содержимое элемента `items[i]` по-прежнему разное для разных `kind` — узел, окно и пара это структурно разные вещи, объединять их в одну форму смысла не было: либо теряешь данные, либо заводишь кучу пустых полей.

Метод рендеринга у всех трёх типов совпадений (`FindMatch`, `ContextWindowMatch`, `DistancePairMatch`) теперь одной формы — `render(return_items, source_path)`, и `DslQueryExecutionResult.to_dict()` вызывает его одинаково для всех. У `DistancePairMatch.render()` раньше была другая сигнатура (без аргументов вообще), и `RETURN` она не учитывалась.

1.5 Семантическая валидация

`src/dsl/execution/query_validator.py:35--58`, метод `_validate_kind_specific_fields`. Раньше было, что каждый вид запроса — свой класс, теперь проверяется явно, одним методом:

kind	Проверка
любой	<code>LIMIT_PAIRS</code> допустим только для <code>DISTANCE</code> (строка 46).
<code>DISTANCE</code>	обязателен <code>TO</code> -селектор; ровно один <code>RETURN distance(entity)</code> .
<code>CONTEXT</code>	обязателен хотя бы один паттерн <code>FOR</code> и указана длина окна.
<code>FIND</code>	<code>RETURN distance(entity)</code> запрещён (строка 57)

Проверку имён сущностей (`_collect_entities`) тоже свёл в один проход по общим полям `Query` (`source`, `target`, `patterns`, `within`, `where`, `returns`) — раньше это было три почти одинаковых куска кода под каждый класс.

1.6 Найденный и исправленный дефект: `WITHIN` у `FIND`

Пока переносил диспетчеризацию исполнения на единый `Query`, заметил: `WITHIN` у `FIND` в исходной системе разбирался и проходил проверку имён, но на результат никак не влиял. Диспетчер (`_dispatch_find`) отдавал воркеру только узел-кандидат и запрос, а воркер (`on_ready_evaluate_find_candidate_request`) смотрел только на `query.where`:

```
matched = query.where is None or self._evaluator.evaluate(message.node,
    query.where)
```

То есть `FIND sentence WITHIN paragraph[=1] RETURN text` возвращал те же предложения, что и без `WITHIN`, — фильтр просто ничего не делал. В единой грамматике `WITHIN` — часть общего хвоста для всех трёх видов запроса, и по одной и той же грамматике никак не видно, что у `FIND` он не работает. Оставляять так не стал, исправил (`src/dsl/actors/execution/execution_worker_actor.py:42--45`) — добавил ту же проверку числа контейнеров-предков, что уже была у `CONTEXT`, только для одного узла:

```
matched = (
    self._within_constraints_satisfied([node], query)
    and (query.where is None or self._evaluator.evaluate(node, query.
        where))
)
```

Проверил двумя тестами (`tests/dsl/test_api_integration.py`): `test_actor_dsl_engine_enforces_within_for_find` — `WITHIN word[=1]` у `FIND sentence` теперь отсекает вообще все совпадения, потому что `word` не предок `sentence` в дереве `document_ast`; `test_actor_dsl_engine_find_within_paragraph_keeps_matching_candidates` — структурно выполнимое ограничение (`WITHIN paragraph[=1]`) число совпадений не меняет, как и должно быть.

1.7 Актёрный конвейер разбора

`src/dsl/actors/parsing/query_parser_actor.py`. Пошаговый разбор по сообщениям тоже был несимметричным: у `CONTEXT` — три промежуточных состояния, у `FIND` — два, у `DISTANCE` — ни одного. С общим хвостом эту асимметрию получилось убрать: теперь у всех трёх видов одинаковые два состояния — `PARSING_HEAD` (`parse_query_head()`) и `PARSING_TAIL` (`parse_query_tail()`). Состояний парсера (`DslQueryParserState`) стало четыре вместо семи, сообщений конвейера разбора — два вместо шести `Continue*`.

1.8 Проверка работоспособности

Тесты гонял из `task-3-1` с `PYTHONPATH=src`. На нетронутой копии исходного кода — 90 из 91 теста. Один падает и без моих правок — `test_cli_noninteractive_smoke_outputs_ast_json`, из-за кодировки консоли Windows (`cp1252`) при выводе кириллицы в `main.py`, к DSL отношения не имеет.

После объединения — 95 из 96 (тот же посторонний сбой плюс пять новых тестов). Что поменялось в тестах:

- `tests/dsl/test_lexer_and_parser.py` — везде вместо `ContextQuery/FindQuery/DistanceQuery` теперь `Query(kind=...)`. Добавил тест на новый порядок `WITHIN/WHERE` у `FIND` и тест на то, что `LIMIT_PAIRS` у `FIND` парсится (отклоняется позже, валидатором).
- `tests/dsl/test_execution_helpers.py` — результаты и сериализация проверяются через `DslQueryExecutionResult`; добавил тесты на новые проверки `QueryValidator` (`LIMIT_PAIRS` вне `DISTANCE`, `RETURN distance(...)` у `FIND`).
- `tests/dsl/test_api_integration.py`, `tests/integration/test_ga_tex_dsl_examples.py`, `tests/integration/test_tex_end_to_end.py` — `payload["type"]` везде теперь `"dsl_query_execution_result"` плюс отдельная проверка `payload["kind"]`; добавил два теста на `WITHIN` у `FIND`.